Padrões GRASP – Acoplamento

Sabrina de F. Souto.

Acoplamento (definição)

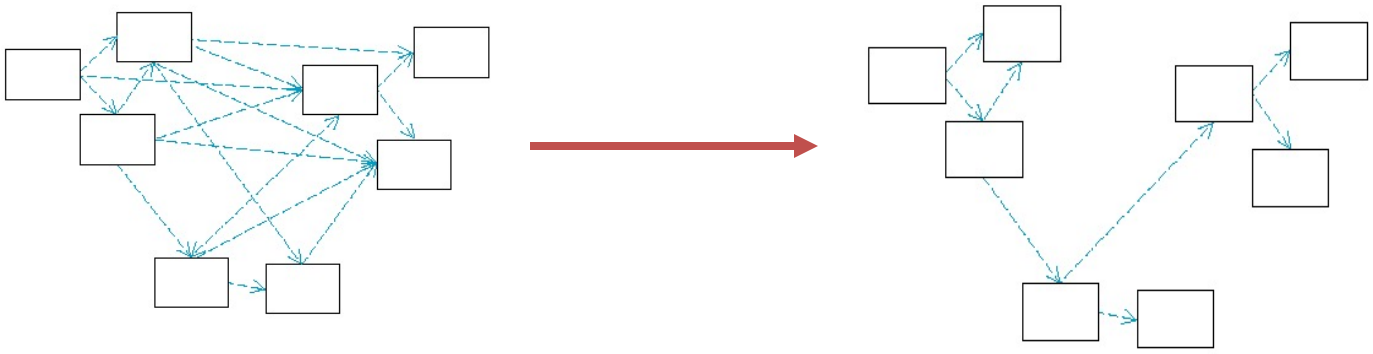
- É uma **MEDIDA** de quão fortemente uma classe está conectada, possui conhecimento ou depende de outra classe
  - X tem um atributo que referencia uma instância de Y
  - X tem um método que referencia uma instância de Y
    - Pode ser parâmetro, variável local, objeto retornado pelo método
  - X é uma subclasse direta ou indireta de Y
    - A herança é um tipo de acoplamento particularmente forte
  - X implementa a interface Y

Baixo Acoplamento

- O que fazer?
  - Tornar os módulos tão independentes quanto possível.
    - Alteração em um módulo tem mínimo de risco de se ter que alterar outros
- O que não fazer?
  - Criar alguns poucos objetos monstruosos (*God classes*) → pouca coesão

Baixo Acoplamento

- Como obter?
  - Eliminando-se comunicação desnecessária
  - Reduzindo-se o número de comunicações necessárias
  - Enfraquecendo a dependência das relações necessárias

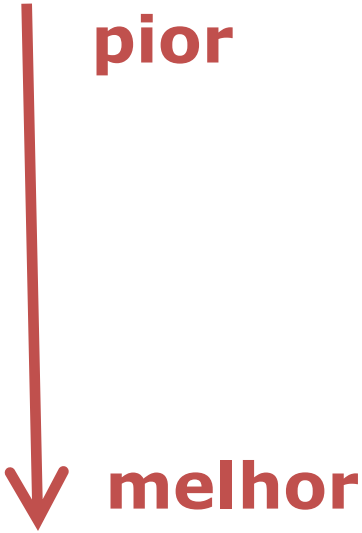


Alto Acoplamento – Implicações

- Classes difíceis de reusar isoladamente
  - Sistema torna-se denso
    - Difícil de entender e de manter
- Leva a sistemas **monolíticos**
  - Não se pode remover ou mudar uma classe sem compreender e mudar muitas outras

Tipos de Acoplamento

- De dados internos
- De dados globais
- De controle
- De dados



Acoplamento de Dados Internos

- Um objeto altera (faz referência) aos dados locais (internos) de um outro objeto

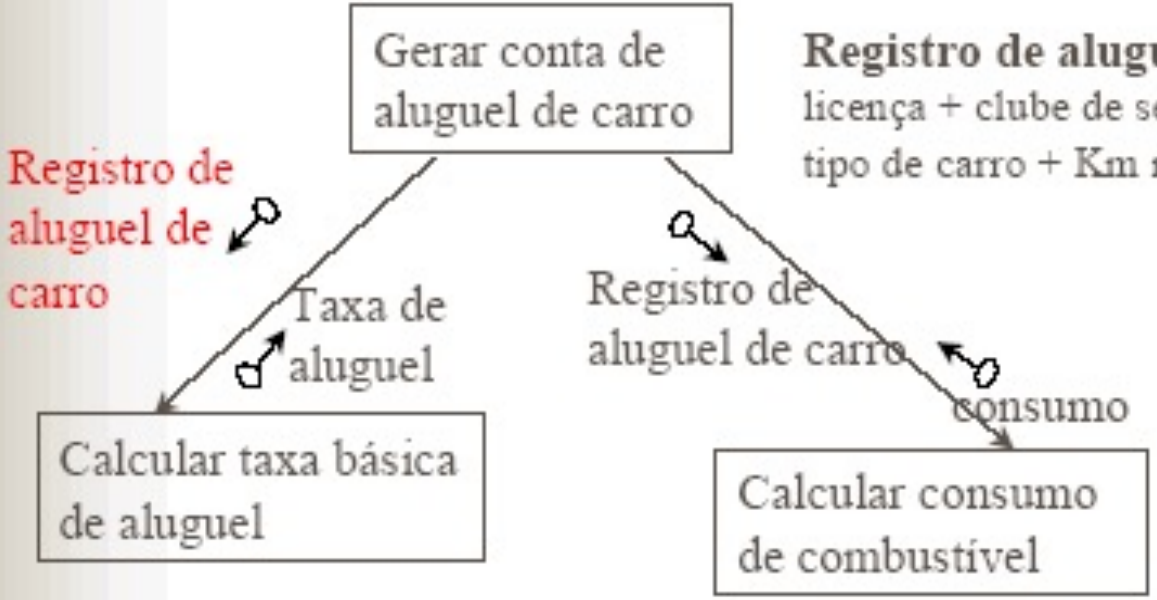
- Exemplo:
  - Se um módulo desvia a seqüência de instruções para dentro de outro (go to)
  - Se um módulo altera um comando do outro

- Tal acoplamento torna o conceito de caixa preta sem sentido.

Acoplamento de Dados Globais

- Dois ou mais objetos compartilham estruturas de dados globais
  - O módulo recebe mais dados do que necessita.
  - Qualquer alteração na estrutura afetará todos os módulos que a utilizam

Acoplamento de Dados Globais

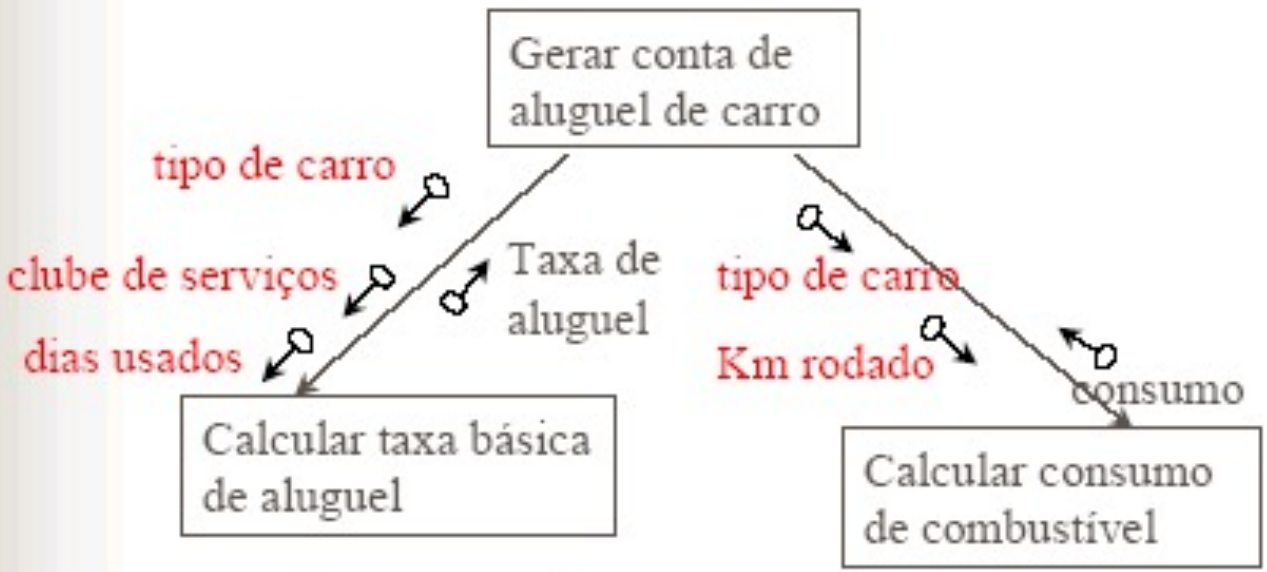


Registro de aluguel de carro = no. da licença + clube de serviço + tipo de combustível + tipo de carro + Km rodados + dias de uso

- Calcular taxa básica de aluguel → requer: clube de serviço+tipo de carro+dias de uso
- Calcular consumo de combustível → requer: tipo de carro + Km rodados

Problema → Alterações nos outros campos → alterações nesses módulos

Acoplamento de Dados Globais



Gerar conta de aluguel de carro X Calcular taxa básica de aluguel

Gerar conta de aluguel de carro X Calcular consumo de combustível

acoplamento por dados

Acoplamento de Controle

- Passar *flags* de controle entre objetos de forma que um objeto controle as etapas de processamento de outro objeto

- Ocorrência comum 1:
  - Objeto **A** manda uma mensagem para objeto **B**
  - **B** usa um parâmetro da mensagem para decidir o que fazer

Acoplamento de Controle

```
public class Lampada {
    public final static int ON = 0;
    public void setLampada(int valor) {
      if(valor == ON) { // liga lampada }
      else if(valor == 1) { // desliga lampada
      } else if(valor == 2) { // pisca }
    ...
 }

Lampada lampapa = new Lampada();
lampada.setLampada(Lampada.ON);
lampada.setLampada(2);
```

PROBLEMAS???

Acoplamento de Controle

```
public class Lampada {
    public void on() {//liga lampada }
   public void off() {//desliga lampada }
    public void pisca() {// pisca}
}

Lampada lampada = new Lampada();
lampada.on();
lampada.pisca();
```

Acoplamento de Controle

- Ocorrência comum 2:
  - Objeto **A** manda mensagem para objeto **B**
  - **B** retorna informação de controle para **A**
- Exemplo: retorno de código de erro

Acoplamento de Controle

```
public class Teste {
    public int printFile(File aImprimir) {
    if(aImprimir está corrompido ) {
            return CORRUPTFLAG;
     ...
}

Teste umTeste = new Teste();
int resultado = umTese.printFile(miniTeste);
if(resultado == CORRUPTFLAG) {
   // fazer algo
} else if(resultado == -243) {
   // etc. etc.
```

Acoplamento de Controle

```
class Teste {
    public int printFile(File aImprimir) throws PrintExeception {
     if(aImprimir.isCorrompido()) {
          throw new PrintExeception();
     ...
}

try {
    Teste umTeste = new Teste();
   umTeste.printFile(miniTeste);
} catch(PrintException printError) {
    // fazer alguma coisa
}
```

Acoplamento de Dados

- Situações
  - Saída de um objeto é entrada de outro
  - Uso de parâmetros para passar itens entre métodos

- Ocorrência comum:
  - Objeto **A** passa objeto **X** para objeto **B**
  - Objeto **X** e **B** estão acoplados
  - Uma mudança na interface de **X** pode implicar em mudanças a **B**

Acoplamento de Dados

```
public class Aluno {
    private String nome;
    private long matrícula;
    public String getNome() { return nome; }
    public long getMatrícula() { return matrícula;}
}

ListaOrdenada listaDeAlunos = new ListaOrdenada();
listaDeAlunos.add(new Aluno(...));
```

Acoplamento de Dados

```
public class ListaOrdenada {
    private Object[] elementosOrdenados = new Object[tam];

  public void add(Aluno x) {
    long matrícula1 = x.getMatrícula();
    long matrícula2 = elementosOrdenados[k].getMatrícula();
    if(matrícula1 < matrícula2) {
        // faça algo
    } else {
        // faça outra coisa
    ...
}
```

Acoplamento de Dados

- Qual o problema com o exemplo anterior?
  - Forte Acoplamento
  - ListaOrdenada sabe muita coisa de Aluno
    - A comparação de alunos é feito com a matrícula
    - A matrícula é obtida com getMatrícula()
    - Matrículas são long (representação de dados)
    - Como comparar matrículas (com <)
  - O que ocorre se mudarmos qualquer uma dessas coisas?

Acoplamento de Dados

```
public class ListaOrdenada {
    private Object[] elemOrd = new Object[tam];

   public void add(Aluno x) {
     if(x.compareTo(elemOrd[K]) < 0) {
          // faça algo }
     else {
          // faça outra coisa
    ...
}
```

Acoplamento de Dados

- Reduzimos o acoplamento escondendo informação atrás de um método
- Problema?
  - ListaOrdenada só funciona com Aluno
- Solução Alternativa
  - use interfaces para desacoplar mais ainda

Acoplamento de Dados

```
public interface Comparable {
    public int compareTo(Object outro);
}

public class Aluno implements Comparable {
    public int compareTo(Object outro) {...}
}

public class ListaOrdenada {
    private Object[] elemOrd = new Object[tam];
    public void add(Comparable x) {
     if(x.compareTo(elementosOrdenados[K]) < 0) {
          // faça algo
     } else {
          // faça outra coisa }
}
```

Baixo Acoplamento

- Uma classe fracamente acoplada não é afetada (ou pouco afetada) por mudanças em outras classes
- Simples de entender isoladamente
- Reuso mais fácil

Dúvida?